

Java Interview Preparation Notes

Core concepts, common interview questions, and explanatory notes for placement preparation

The Java Virtual Machine (JVM)

The Java Virtual Machine is the runtime engine responsible for executing compiled Java bytecode on any platform, which is the basis of Java's well known write-once-run-anywhere capability. When a Java program is compiled, the source code in a .java file is converted by the compiler into bytecode stored in a .class file, and this bytecode is what the JVM interprets or compiles just-in-time into native machine instructions for the underlying operating system and hardware. Interview questions frequently probe whether a candidate understands that the JVM itself is platform dependent even though the bytecode it executes is platform independent, since each operating system requires its own JVM implementation.

A closely related interview topic is memory management inside the JVM, particularly the division between the heap and the stack. The heap stores objects and is further divided into generational spaces such as the young generation and old generation, which is directly relevant to how garbage collection operates. The stack, in contrast, stores method call frames, local variables, and partial results, and is organized per thread. Understanding this distinction helps explain common interview questions about memory leaks, stack overflow errors, and the behavior of garbage collection algorithms such as mark-and-sweep or generational garbage collection.

Object Oriented Programming Concepts

Object oriented programming in Java rests on four pillars, and interviewers typically expect candidates to explain each with a concrete example rather than a textbook definition alone. Encapsulation refers to bundling data and the methods that operate on that data within a single unit, typically enforced in Java through private fields accessed via public getter and setter methods, which protects the internal state of an object from unintended external modification.

Abstraction, discussed further below, focuses on exposing only essential behavior while hiding implementation detail, and is often confused with encapsulation during interviews, so it is worth being able to clearly separate the two concepts when answering. A strong answer distinguishes abstraction as a design level concept concerned with what an object does, from encapsulation as an implementation level concept concerned with how an object's internal state is protected.

Inheritance

Inheritance allows a subclass to acquire the fields and methods of a superclass, promoting code reuse and establishing an is-a relationship between classes. Java supports single inheritance for classes, meaning a class can extend only one other class directly, but supports multiple inheritance of type through interfaces. A frequently asked interview question involves explaining why Java does not support multiple inheritance for classes, with the standard answer citing the diamond problem, where ambiguity would arise if two parent classes defined the same method and the compiler could not determine which implementation the subclass should inherit.

Method overriding, which is closely tied to inheritance, allows a subclass to provide a specific implementation of a method already defined in its superclass, and is resolved at runtime through dynamic method dispatch. This is often tested alongside method overloading, which is resolved at compile time

based on method signature, and interviewers commonly ask candidates to explain the difference with a short code example illustrating each.

Polymorphism

Polymorphism allows objects of different classes to be treated as objects of a common superclass, most commonly demonstrated in Java through upcasting a subclass reference to a superclass type and observing that the overridden method of the actual runtime object is invoked rather than the method defined in the reference type. This behavior, known as runtime polymorphism, is a favorite interview topic because it directly connects to how the JVM resolves method calls using a mechanism called the virtual method table.

Compile time polymorphism, achieved through method overloading, is the other form typically discussed, and candidates are expected to be able to identify which form of polymorphism a given code snippet demonstrates when presented with one during a live coding interview.

Abstraction

In Java, abstraction is implemented primarily through abstract classes and interfaces. An abstract class can contain both abstract methods without a body and concrete methods with an implementation, and cannot itself be instantiated, whereas an interface historically could only declare method signatures, though modern versions of Java allow default and static methods within interfaces as well. A common interview question asks when a candidate would choose an abstract class over an interface, and a well rounded answer notes that abstract classes are appropriate when related classes share common state or behavior, while interfaces are preferred when unrelated classes need to guarantee a particular capability without sharing an inheritance hierarchy.

Exception Handling

Java's exception handling model divides throwable conditions into checked exceptions, which must be either caught or declared in a method's throws clause, and unchecked exceptions, which extend RuntimeException and do not require explicit handling at compile time. Interview questions in this area often ask candidates to explain the difference between these two categories and to justify why certain exceptions, such as IOException, are checked while others, such as NullPointerException, are unchecked.

The try, catch, finally, and try-with-resources constructs are also common topics, particularly the guarantee that code in a finally block executes regardless of whether an exception was thrown, and the way try-with-resources automatically closes resources implementing the AutoCloseable interface, which reduces the risk of resource leaks compared to manual closing in a finally block.

Collections Framework

The Java Collections Framework provides a unified architecture for storing and manipulating groups of objects, and interview preparation in this area typically centers on understanding the differences between List, Set, and Map implementations. ArrayList and LinkedList are commonly compared, with ArrayList offering fast random access due to its underlying array structure and LinkedList offering faster insertion and deletion in the middle of the list due to its doubly linked node structure.

HashSet, LinkedHashSet, and TreeSet are typically contrasted based on ordering guarantees and underlying data structures, with HashSet offering no ordering guarantee but constant time average performance through hashing, LinkedHashSet preserving insertion order, and TreeSet maintaining sorted order through an underlying red-black tree. Similarly, HashMap, LinkedHashMap, and TreeMap are compared on the same basis, and candidates are often asked to explain how HashMap handles collisions internally, typically through chaining or, in more recent Java versions, through a hybrid approach that converts a bucket to a balanced tree once it grows beyond a certain threshold.